

When interviewer asks:
“How do you monitor systems?”

Say:

- Metrics → Prometheus
- Logs → Loki
- Visualization → Grafana
- Alerting → Alertmanager (optional next step)

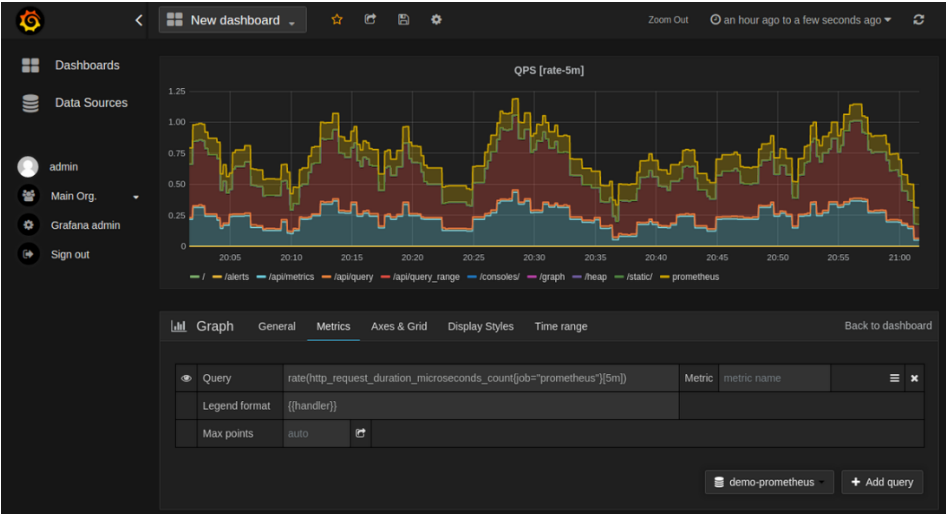
What is Prometheus?

Prometheus = a tool that collects numbers (metrics)

“Prometheus uses a pull-based monitoring model to collect metrics and store them as time series. It uses time series data to record metrics operations that change over time”

Prometheus keeps the time series database.

Imagine you check your **body temperature every hour** and write it in a notebook.
The notebook is Prometheus.



Components of Prometheus-

- Prometheus Server - It scrapes and stores time series data.
- Pushgateway - It collects metrics from short-lived jobs.
- Prometheus Targets - Instrumented jobs from which Prometheus scrapes metrics.
- Alertmanager - It handles alerts.

Grafana

Grafana = a tool that shows data in graphs.

You turn your boring notebook into:

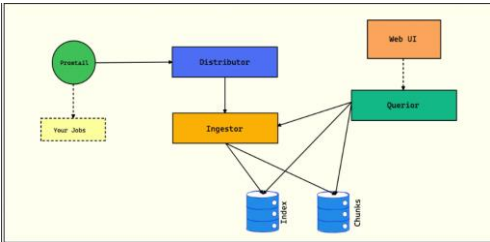
- colorful graphs
- easy-to-understand pictures



Loki –

Loki is a tool which is used to read the logs(Messages) for example I want only specific logs where my services are failing.
example – Database connection failed.

Loki Arch –



Components of Loki –

Promtail – It is used to scrob the container of Kubernetes.

Distributor – It distribute the logs between the Ingestors.

Ingestor – In this step it takes the logs in chunks and indexes them in time series database.

Querier – In this step the query is done. And output is exported from chunks.

🔗 **Prometheus** → numbers (metrics)

🔗 **Grafana** → graphs (visuals)

🔗 **Loki** → logs (messages)

Now lets build a project

What is the difference between

Loki and Prometheus are both open source tools. While Loki is a log aggregation tool, Prometheus is a metrics monitoring tool. Loki’s design is inspired by Prometheus but for logs.

Feature	Prometheus	Loki
Data type	Numbers (metrics)	Text (logs)
Example	CPU = 80%	"Server error occurred"
Purpose	Monitor performance	Debug problems
Data format	Time-series data	Log lines
Storage	Stores numeric values over time	Stores log messages
Query	PromQL	LogQL

Service – what they does – port utilized.

Service	What it does	Port
Prometheus	Collects and stores metrics (CPU, memory, requests)	9090
Grafana	Shows data in dashboards (graphs, charts)	3000
Loki	Collects and stores logs (text events, errors)	3100
Promtail	Sends logs to Loki	9080
Node Exporter	Provides system metrics to Prometheus	9100

Lets Build A project

Monitor the Flask app every 15 mins check the key changes inside.

App → Prometheus → Grafana

```
sudo apt update
sudo apt update && sudo apt install python3 python3-pip python3-venv

mkdir my_flask_app && cd my_flask_app

python3 -m venv venv

source venv/bin/activate

pip install Flask

flask --version
```

nano app.py

app.py

```
from flask import Flask
```

```
import time
```

```
from prometheus_client import start_http_server, Counter, Histogram
```

```
app = Flask(__name__)
```

```
REQUEST_COUNT = Counter('request_count', 'Total Requests')
```

```
REQUEST_TIME = Histogram('response_time_seconds', 'Response time')
```

```
@app.route('/')
```

```
def home():
```

```
    REQUEST_COUNT.inc()
```

```
    return "Hi Welcome to Cloud SRE Adda"
```

```
@app.route('/delay')
```

```
@REQUEST_TIME.time()
```

```
def delay():
```

```
    REQUEST_COUNT.inc()
```

```
    time.sleep(5)
```

```
    return "Response after delay"
```

```
if __name__ == '__main__':
```

```
    start_http_server(8000) # metrics port
```

```
    app.run(host='0.0.0.0', port=5000)
```

python3 app.py

🔗 Test:

- <http://<IP>:5000/>
- <http://<IP>:5000/delay>
- <http://<IP>:8000/metrics>

STEP 2: Install Prometheus

```
wget https://github.com/prometheus/prometheus/releases/download/v3.10.0/prometheus-3.10.0.linux-amd64.tar.gz
```

```
tar xvfz prometheus-3.10.0.linux-amd64.tar.gz
```

```
cd prometheus-*
```

```
vi prometheus.yml
```

```
# prometheus.yml
```

```
global:
```

```
scrape_interval: 5s
```

```
scrape_configs:
```

```
- job_name: "flask-app"
```

```
static_configs:
```

```
- targets: ["localhost:8000"]
```

```
./prometheus --config.file=prometheus.yml
```

Check port 9090 – Prometheus UI and pickup variables from there.

“search fiels from 9090”

Now It’s integrate grafana.

STEP 3: Install Grafana

```
sudo apt update && sudo apt upgrade -y
```

```
sudo apt install -y apt-transport-https software-properties-common wget
```

```
echo "deb https://packages.grafana.com/oss/deb stable main" | sudo tee /etc/apt/sources.list.d/grafana.list
```

```
sudo apt update
```

```
sudo apt install grafana -y
```

```
sudo systemctl start grafana-server
```

```
sudo systemctl enable grafana-server
```

```
sudo systemctl status grafana-server
```

Go to the IP:3000

login

Add visualizatiion → Create a new datasource